

4-5 · 카탈로그 연결 및 레이크하우스 통합 · 정리·회고 0.5H

주차 회고 · 산출물 점검

데이터를 한 곳에 두고 도구를 갈아 끼운다 — 4주차

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① 4-1~4-5 에서 배운 것과 남은 결과물을 한 표로 본다

② 체크리스트로 4주차 산출물을 스스로 점검한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ 볼륨은 남기고 컨테이너를 정지해 5주차를 준비한다

한 주를 관통한 생각 — 데이터를 한 곳에 두고 도구를 갈아 끼운다

닷새 — 배운 것과 남은 결과물

차시	배운 것	남은 결과물
4-1	오브젝트 스토리지와 S3 API	MinIO · raw·warehouse 버킷
4-2	Iceberg 메타데이터 사슬과 타임 트래블	orders 테이블 · 스냅샷 이력
4-3	랜딩 존 → 정제 → MERGE 먹등 적재	etl_orders.py
4-4	Trino 로 같은 테이블 조회	iceberg.properties · daily_sales
4-5	카탈로그 중심 상호운용 검증	아키텍처 다이어그램 · 쿼리 스크립트

3주차에서 도구를 목적에 따라 골랐다면, 4주차는 그것을 저장소 수준에서 가능하게 만드는 구조

5주차로 이어지는 것

오늘 그린 그림의 왼쪽에 붙는다

1 Spark

이번 주는 spark-sql 로 가볍게 — 5주차엔 클러스터를 띄우고 드라이버·익스큐터·서플

2 Iceberg

5주차 Spark 실습에서도 이번 주 테이블을 그대로 쓴다

3 파티셔닝

3-3 에서 맛보고 4주차에서 개념 — 5-5 에서 성능 문제로 다시 만난다

4 아키텍처 그림

왼쪽에 6주차 Kafka, 8주차 Airflow 가 붙는다

여덟 개를 스스로 확인해 보세요

- 1 카탈로그가 단일 진실 원천인 이유를 설명할 수 있다
- 2 카탈로그가 없으면 깨지는 것을 세 가지 이상 말할 수 있다
- 3 REST·Hive Metastore·Glue·Nessie 의 차이를 대략 말할 수 있다
- 4 Trino INSERT → Spark 조회 · Spark 스키마 변경 → Trino 를 양방향 검증했다
- 5 두 엔진의 스냅샷 개수까지 일치하는 것을 확인했다
- 6 카탈로그 이름은 별명 · uri·warehouse 가 실체임을 실험으로 확인했다
- 7 다이어그램에 구성 요소 5개와 포트를 표기했다
- 8 lakehouse_queries.sql 이 끝까지 돌고 down→up 뒤에도 데이터가 남는다

증상으로 찾는 원인과 해결

증상	원인	해결
Spark 테이블이 Trino 에 안 보임	두 엔진이 다른 카탈로그 (uri 또는 warehouse)	rest:8181 · s3://warehouse/ 양쪽 값 대조
넣은 데이터가 다른 쪽에 안 보임	쓰기가 실패·미완료 커밋 전엔 어디서도 안 보임	성공 메시지 확인 → \$snapshots 새 스냅샷 확인
FOR VERSION AS OF 스냅샷 못 찾음	다른 테이블의 id 또는 만료된 스냅샷	같은 테이블 \$snapshots 결과에서 id 복사
catalog 파일 넣었는데 Trino 가 못 읽음	컨테이너는 기동 시점에 카탈로그를 읽는다	restart trino 뒤 40초 안팎 대기

「안 보인다」면 순서는 늘 같다 — 카탈로그 설정 대조 → 커밋 여부("orders\$snapshots") 확인

자주 나오는 질문 네 가지

1 실무에서도 이 구성을 그대로 쓰나요?

구성 요소는 같고 규모만 다르다 — MinIO 자리에 S3, REST 대신 Glue, 컨테이너 하나 대신 여러 대

2 카탈로그가 죽으면?

데이터 파일은 남아 있으니 카탈로그를 복구하면 된다 — 그래서 카탈로그도 백업 대상

3 왜 Spark 와 Trino 를 둘 다 쓰나요?

하나만 써도 된다 — 무거운 적재와 빠른 조회는 요구가 달라 커질수록 나누는 편이 유리

4 5주차에 이 환경을 계속 쓰나요?

5주차는 Spark 클러스터를 따로 띄운다 — 메모리가 빠듯하면 stop, 볼륨은 남는다

볼륨은 남기고 정지만 — 5주차에도 쓴다

1

docker compose stop

down -v 는 볼륨까지 지운다 — 이번 주 Iceberg 테이블이 전부 사라지니 쓰지 않는다

2

docker compose ps -a · docker volume ls

grep -E "minio|lakehouse" 로 볼륨이 남아 있는지 확인

3

mkdir -p ~/data-eng/week5

5주차 실습 폴더를 미리 만들어 둔다 — 메모리가 부족하면 stop, 필요할 때 start

쓰면 안 되는 것

down -v

볼륨 삭제

다음 회차 예고

5-1 · 분산 처리와 Spark 아키텍처

01

이론 1H

분산 데이터 처리와
Spark 개요

02

실습 1.5H

단일 노드 Spark
PySpark 첫 실행

03

실습 1H

Spark UI 로
잡 구조 읽기

04

정리 0.5H

정리·회고
이론·실습

준비: Docker Desktop 실행 · docker version 정상 출력 확인 · 4주차 MinIO·Iceberg 컨테이너는 내려 두어도 됨(5-3 에서 다시 씀)

4-5 · 정리·회고 0.5H 마무리

다음 회차는 5-1 분산 처리와 Spark

한 대로 안 되면 여러 대에 나눠 맡깁니다 — 드라이버·익스큐터 구조와 지연 평가를 한 대 안에서 관찰합니다

✓ 스케일업과 스케일아웃의 차이

✓ 드라이버 · 익스큐터 · 클러스터 매니저

✓ transformation 과 action (지연 평가)

✓ Spark UI 의 Job → Stage → Task

준비할 것

docker version 확인 · week5 폴더